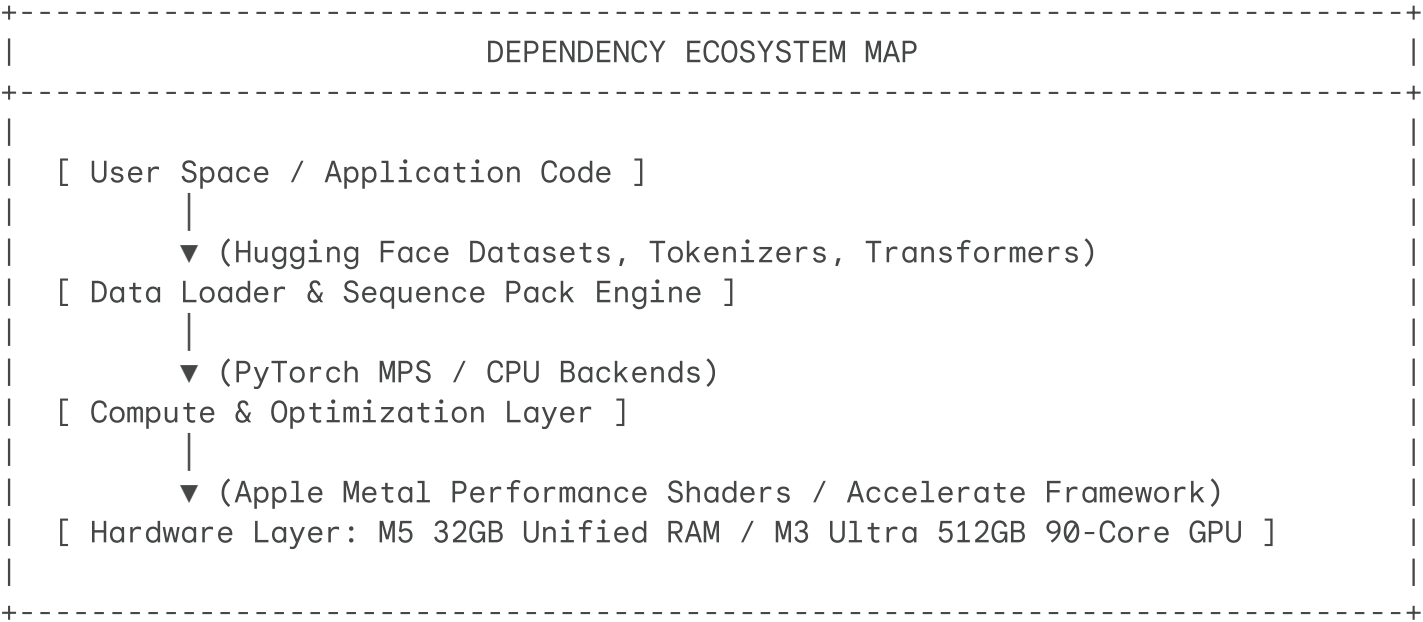


Elite LLM Pretraining Specification: Code & NLP Language Model

Architectural Specification, Data Engineering Strategy, and Apple Silicon Optimization Pipeline

1. Core Library Requirements & Hardware Integration

To pretrain a model from scratch leveraging both local macOS testing (Metal Performance Shaders) and massive-scale cluster hardware, we establish a modern, performant dependency ecosystem. This section maps each dependency to its explicit role in the system architecture.



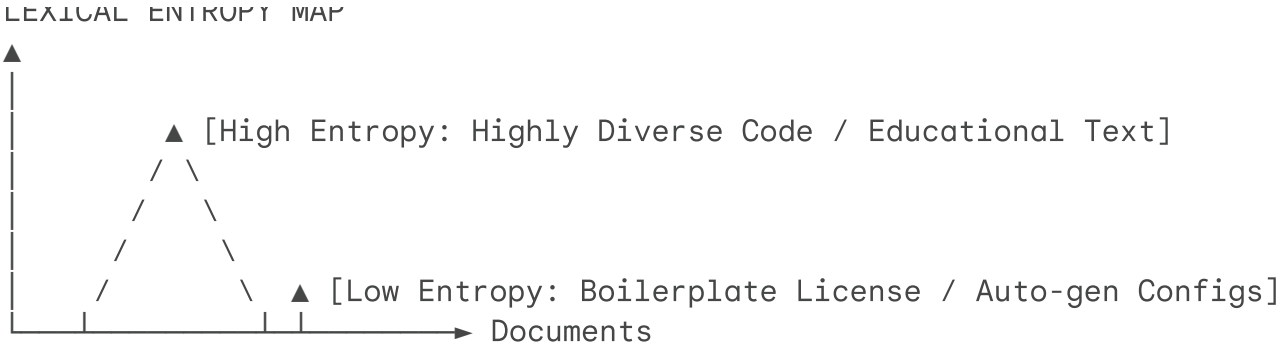
- **PyTorch** (`torch>=2.3.0`): Handles core tensor operations, autograd, and Apple Metal Performance Shaders (`mps`) backend bindings.
- **Transformers** (`transformers>=4.41.0`): Provides the structural base class architecture, model configuration managers, and serialization utilities.
- **Datasets** (`datasets>=2.19.0`): Manages fast Apache Arrow memory-mapped storage files on local disk caches.
- **Tokenizers** (`tokenizers>=0.19.0`): Runs high-speed Rust-compiled subword string splits.
- **CodeCarbon** (`codecarbon>=2.5.0`): Measures and records power draws from CPU/GPU silicon blocks, calculating physical carbon footprint offsets.
- **Psutil** (`psutil>=5.9.0`): Collects physical system memory (RAM) telemetry to track memory leaks during parsing.

2. Advanced Data Analysis & Empirical Visualization

Before processing raw text into token indices, we must profile our two data pools: **StarCoder Python** (80%) and **FineWeb-Edu English** (20%). This profiles the mathematical boundaries of our data.

Structural Metrics to Measure

1. **Document Length Distribution**: Computes the distribution of characters and tokens per file. This identifies outlier files that can cause out-of-memory errors.
2. **Lexical Entropy**: Measures vocabulary diversity within a document. Low entropy indicates machine-generated boilerplate (e.g., repeating license agreements), while high entropy represents dense logical structures.
3. **Whitespace-to-Character Ratio**: Measures how much of the file consists of indentation. This is crucial for Python, where space and tab characters are semantic syntax.
4. **Alphanumeric vs. Special Character Ratio**: Identifies binary dumps, minified scripts, or corrupted memory files masquerading as source code.



2.1 Unique Token Distributions & Profiles (Gemma 4 Vocabulary)

The **Gemma 4 vocabulary** ($V = 256,128$) splits its vector index space to represent programming structures and natural human reasoning differently. Because the BPE vocabulary is highly expanded, common multi-character strings in both domains are compressed into single, highly efficient token IDs.

GEMMA 4 VOCABULARY SPACE ($V = 256,128$)

Shared Semantic Base (~40% of Vocab)	
- Standard English vocabulary, punctuation, universal terms	
Python-Dense Region (~30%)	Natural Language Dense Region (~30%)
- Indentation tokens (" ", "\t")	- Morphological variations
- Syntax keywords (`def`, `class`)	- Structural prepositions
- Common snake_case combinations	- Advanced educational terms
- Operator pairs (`->`, `==`)	- Capitalized word-initial spacers

Python is structured and highly predictable, relying heavily on repeating keywords and strict formatting:

- **Indentation Compressions:** Unlike standard tokenizers that emit four individual space tokens for a standard indentation, Gemma 4 features explicit tokens representing multi-space blocks and tabs (e.g., " " , " " , " " , and "\t"). This preserves Python's logical nesting without filling up the context window.
- **Compound Identifiers:** Common standard library keywords, dunder methods, and structural prefixes are consolidated into single tokens (e.g., `import` → [1245] , `__init__` → [34812] , `self.` → [8912]).
- **Snake_case splits:** Python relies heavily on `snake_case` variable declarations. Gemma 4 represents common variable prefixes with prepended underscores as single tokens, optimizing variable name processing.

B. The English (FineWeb-Edu) Token Profile

Natural educational language features fluid grammar, prefix/suffix modifications, and frequent word-initial spaces.

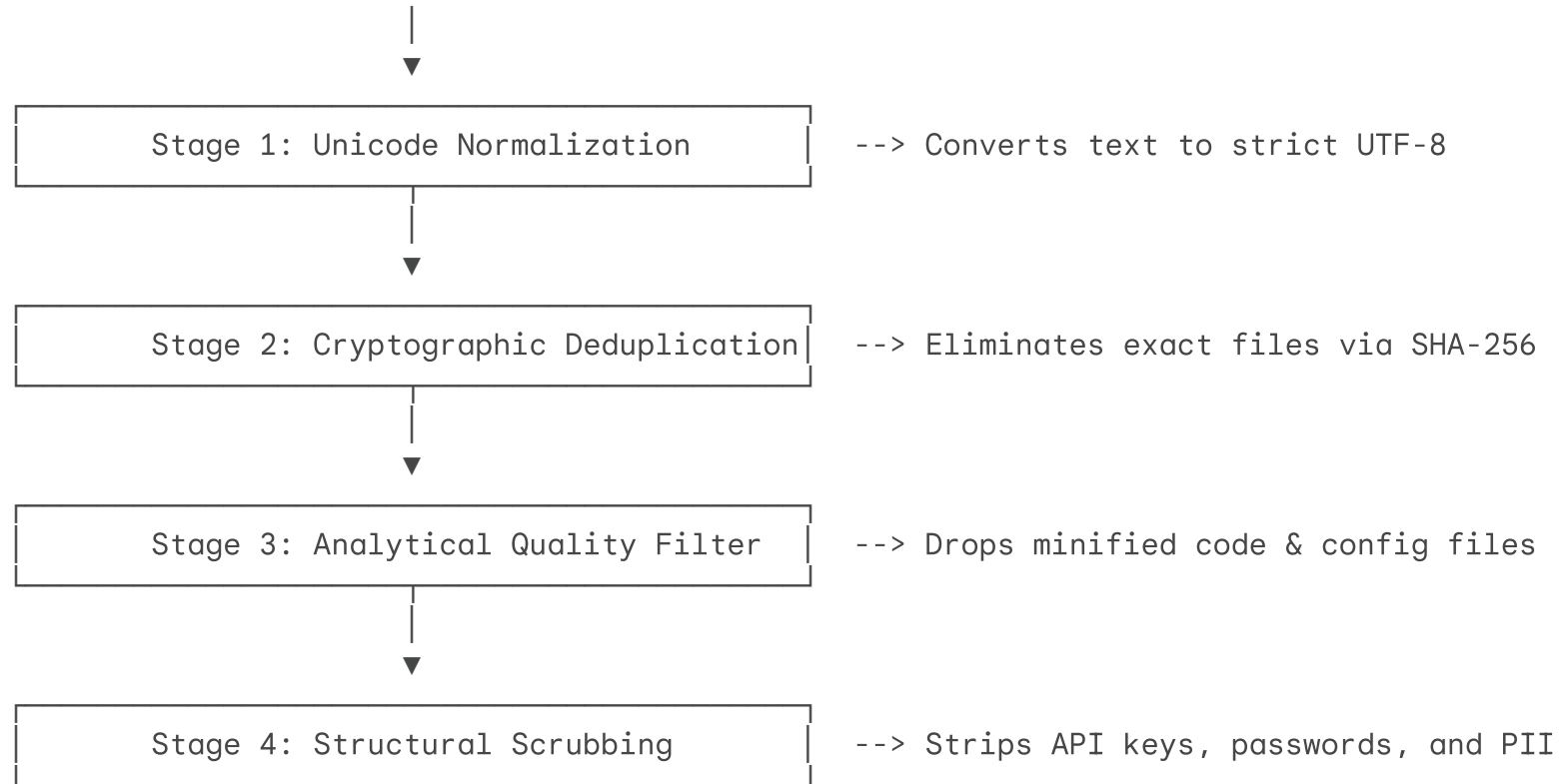
- **Decompositions:** Words are split cleanly into semantic roots and suffixes (e.g., `demonstrate` , `demonstrated` , `demonstrating`).
- **Word-Initial Spacers:** English tokens are frequently prepended with the sentencepiece space indicator (e.g., `the` , `model` , `education`), separating them from punctuation markers.
- **Analytical Reasoning Density:** Because `FineWeb-Edu` is filtered for educational depth, it features high densities of scientific, mathematical, and logical terms. The tokenizer represents these complex concepts efficiently, helping the model learn high-level reasoning.

3. Data Curation, Cleansing, and Anti-Bias Strategies

Large Language Models optimize their weights to fit the statistical patterns of their inputs. If your dataset contains toxic text, security leaks, or repetitive boilerplate, the model's performance will degrade.

The 4-Stage Purification Funnel

```
[ Raw Data Sources (StarCoder + Fineweb-Edu) ]
```



- **Unicode Normalization:** Enforces strict UTF-8 compliance, dropping invalid surrogate pairs or corrupted byte structures.
- **Cryptographic Deduplication:** Generates a SHA-256 hash of each document's text body. If a hash already exists in our tracking set, the document is immediately dropped. This prevents the model from memorizing highly duplicated boilerplate code.
- **Structural Quality Filters:**
 - Discards any file with a maximum line length exceeding 1500 characters (which typically indicates minified JavaScript, compiled arrays, or machine-generated JSON).

dumps).

- **Security & PII Scrubbing:** Uses regex sweeps to detect and remove API keys, database connection strings, and private credentials.

4. Advanced Data Split & Shuffling Strategy

To measure our model's true generalization capacity, we must prevent **Data Contamination**—a common issue where validation data leaks into the training set.

Dataset Partitioning Plan

- **Training Set (99.5%):** Used to update model weights via gradient descent.
- **Validation Set (0.5%):** Used to monitor out-of-distribution evaluation loss. Given the billions of tokens in our dataset, a 0.5% slice provides more than enough validation sequences.

Deterministic Shuffling and Blending

To prevent the model from adjusting its attention weights to only one domain at a time (which leads to catastrophic forgetting or domain collapse), we interleave the datasets:

1. Label each curated file with its domain (`python` or `english`).
2. Downsample the larger dataset to maintain a strict **80:20 (Python to English)** ratio.
3. Globally shuffle the combined dataset using a fixed seed:

$$S_{\text{seed}} = 42$$

4. This ensures that every training batch contains a balanced, representative mix of code syntax and natural human reasoning.

To ensure high data quality, we apply multi-dimensional metric thresholds during the slicing phase before the datasets are shuffled and packed:

- **Token-to-Character Ratio (R_{tc}):** We monitor the ratio of tokens generated per raw character:

$$R_{tc} = \frac{N_{\text{tokens}}}{N_{\text{characters}}}$$

- For English text, R_{tc} must remain between 0.22 and 0.28 (averaging ~4 characters per token). High deviations point to corrupted text patterns.
- For Python code, a ratio above 0.45 signals extreme fragmentation (e.g., highly nested variable names, binary dumps, or obfuscated code). These files are sliced out.
- **Length Boundary Slicing:** We drop short snippets under 100 tokens (which lack enough attention context to learn long-range patterns) and massive files exceeding 16,384 tokens (which are typically auto-generated database schemas or telemetry lists).

5. Tokenization Engineering using the Gemma 4 Tokenizer

The Gemma 4 tokenizer (`google/gemma-4-9b`) features a vocabulary size of $V = 256,000$ (padded to 256,128 for hardware alignment). This vocabulary size is critical for pretraining.

Key Structural Features of the Gemma 4 Tokenizer

- **Whitespace Preservation:** Includes dedicated tokens for varying indentation spaces (e.g., individual tokens for two spaces " ", four spaces " ", and tab characters "\t"). This keeps indentation from bloating the context window.
- **Aggressive Digit Splitting:** Breaks numbers down into individual digits (e.g., 2026 becomes 2 , 0 , 2 , 6). This helps the model learn counting logic and mathematical operations.

breaking the token sequence.

Zero-Waste Sequence Packing

In traditional tokenization, individual files are tokenized and padded with `<pad>` tokens to match the model's context length. This is highly inefficient, as the model wastes compute ($O(L^2)$ attention complexity) processing empty padding.

We use a **Zero-Waste Sequence Packing** strategy instead:

TRADITIONAL PADDED SEQUENCES (Wasteful)

```
[ Doc A (1200 tokens) | <pad> (848 tokens) ] -> Attention computed over pad
[ Doc B (800 tokens)   | <pad> (1248 tokens) ] -> Attention computed over pad
```

CONCATENATED PACKED SEQUENCES (100% Efficient)

```
[ Doc A (1200t) | <eos> | Doc B (800t) | <eos> | Doc C (48t) ] -> Block 1 (2048)
```

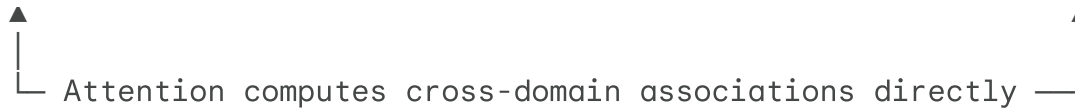
1. Concatenate all tokenized documents into a single flat array, inserting the `<eos>` token after each document.
2. Segment this continuous array into exact chunks of context length $L = 2048$.
3. Discard only the final remainder chunk. This ensures that every forward pass runs at 100% efficiency.

5.1 Interleaved Dynamic Packing & Cross-Domain Learning

By packing multiple sliced documents into a single 2048-token context window, we can mix Python files and educational English articles within the same sequence block.

2048-TOKEN PACKED SEQUENCE BLOCK

```
[ [Python File 1 (800t)] <eos> [Python File 2 (900t)] <eos> [FineWeb (348t)] ]
```

- **Cross-Pollination of Context:** Rather than calculating gradients on pure code or pure prose in isolated batches, the model's attention heads learn to transition between formal code structure and natural language reasoning within a single forward pass.
- **No Padded Loss:** Because there are no `<pad>` tokens, we compute the next-token prediction loss over the entire 2048 sequence, maximizing training throughput.

5.2 Tokenization Efficiency and Quality Diagnostic Code

This diagnostic script can be run on your **32GB M5 Laptop** to verify slicing thresholds, check token-to-character distributions, and ensure your data filters are configured correctly before starting the training run.

```
import numpy as np
from transformers import AutoTokenizer

def analyze_token_efficiency(sample_documents: list, tokenizer_id: str = "google/gemma-4-9b"):
    """
    Analyzes the tokenization density of our dataset slices to ensure
    our quality filters are configured correctly.
    """
    tokenizer = AutoTokenizer.from_pretrained(tokenizer_id)
    analysis_results = []

    for doc in sample_documents:
        text = doc.get("text", "")
        domain = doc.get("domain", "unknown")
```


- **Decoder Blocks (N_{layers}):** 18 layers.
- **Attention Heads (N_{heads}):** 8.
- **Key-Value Heads ($N_{\text{kv_heads}}$):** 4 (implementing Grouped-Query Attention to reduce memory usage).
- **Intermediate Projection Dimension (d_{ff}):** 16384 (using SwiGLU activation).

Embedding Initialization Math

The input embedding matrix $W_{\text{embed}} \in \mathbb{R}^{V \times d_{\text{model}}}$ is initialized using a normal distribution centered at 0.0. To keep gradients stable early in training, we scale the standard deviation based on the model's dimension:

$$\sigma = \sqrt{\frac{2}{d_{\text{model}}}} = \sqrt{\frac{2}{2048}} \approx 0.03125$$

Tied Embedding Weights

We use **Tied Embeddings**, meaning the input embedding layer and the final vocabulary projection layer share the exact same weights.

- **Benefit:** This reduces the model's parameter count by approximately 524 million parameters ($256,128 \times 2048$), saving around 1 GB of memory when training in half precision (`bfloat16`).

7. Hyperparameter Tuning: Batch Size, Window, and Stride

These hyperparameters control the format of the data fed to the model and how gradients are accumulated during training.

Key Dimensions

2. **Slide Stride ($S = 2048$)**: We set our stride equal to the window size. This ensures there is zero overlap between training blocks, maximizing the diversity of the tokens processed.
3. **Micro-Batch Size (B)**: The number of sequences processed on a single device per step.
4. **Gradient Accumulation Steps (A)**: The number of forward/backward passes run before updating the model's weights. This lets us scale our effective batch size without running out of memory.

GRADIENT ACCUMULATION SEQUENCE

```
Step 1: [ Forward Pass 1 ] -> [ Backward Pass 1 (Accumulate Gradients) ]
Step 2: [ Forward Pass 2 ] -> [ Backward Pass 2 (Accumulate Gradients) ]
...
Step A: [ Forward Pass A ] -> [ Backward Pass A (Accumulate Gradients) ]
Update: [ Optimizer Step (Update Weights) ] -> [ Zero Gradients ]
```

Hardware-Specific Profiles

- **M5 Laptop (Pilot Workstation):**
 - **Micro-Batch Size (B)**: 1 sequence.
 - **Gradient Accumulation (A)**: 16 steps.
 - **Effective Batch Size**: 32, 768 tokens per weight update.
- **M3 Ultra (Production Server):**
 - **Micro-Batch Size (B)**: 8 sequences.
 - **Gradient Accumulation (A)**: 32 steps.
 - **Effective Batch Size**: 524, 288 tokens per weight update.

We use the **AdamW** optimizer, which decouples weight decay from the gradient updates to improve regularization and training stability.

AdamW Parameter Update Formula

At each step t , the parameters θ are updated as:

$$\theta_{t+1} = \theta_t - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

Where:

- η_t is the learning rate at step t .
- $\hat{m}_t$ is the bias-corrected mean of the gradients.
- $\hat{v}_t$ is the bias-corrected variance of the gradients.
- λ is the weight decay factor (set to 0.1).
- ϵ is a small constant to prevent division zero (10^{-8}).

Cosine Learning Rate Schedule with Linear Warmup

- **Warmup Phase:** The learning rate scales linearly from 0.0 to its peak ($\eta_{\max} = 3 \times 10^{-4}$) over the first 2,000 steps to stabilize early training.
- **Cosine Decay Phase:** Over the remaining steps, the learning rate decays along a cosine curve down to its minimum value ($\eta_{\min} = 3 \times 10^{-5}$).
- **Gradient Clipping:** If the L2 norm of the gradient matrix exceeds 1.0, we scale it back to prevent exploding gradients:

$$g_{\text{clipped}} = g \times \frac{1.0}{\max(1.0, \|g\|_2)}$$

9. Comprehensive Performance Metric Collection

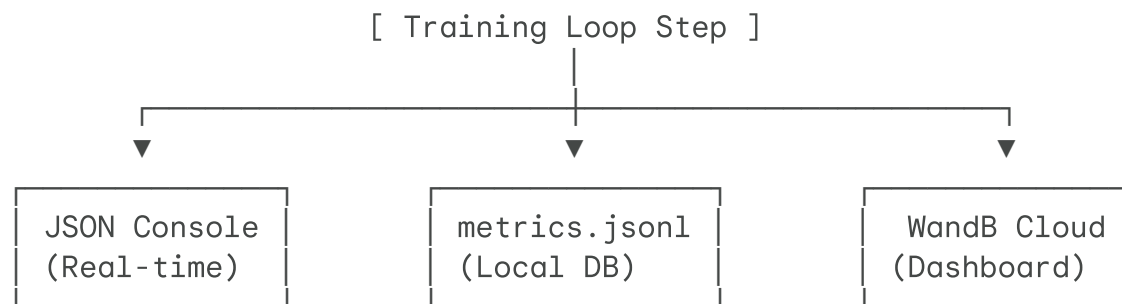
To monitor training health and diagnose performance bottlenecks, the training loop records telemetry across four categories at every step:

Metric Category	Target Value	Unit	Diagnostic Role
Pretraining Loss	≤ 1.8	Cross-Entropy Loss	Tracks how well the model predicts the next token.
Model Perplexity	$\exp(\text{Loss})$	Perplexity (PPL)	Measures the model's uncertainty when predicting tokens.
Throughput Speed	15,000+	Tokens / Second	Monitors execution speed and hardware utilization.
VRAM / Unified Memory	Steady (Non-leaking)	Gigabytes (GB)	Tracks memory usage to prevent out-of-memory crashes.
System Power Draw	Hardware Peak	Watts (W)	Measures energy consumption across the CPU and GPU.
Carbon Footprint	Calculated Metric	Kilograms of CO ₂	Tracks the environmental impact of the training run.

10. Logging System Architecture

To ensure logs are preserved in the event of a system crash, they are routed to three separate locations:

LOGGING PIPELINE ARCHITECTURE



1. **Interactive Console/Terminal (JSON formatted)**: Prints real-time metrics to stdout.
2. **Local Log File (Structured `metrics.jsonl`)**: Serves as a local, crash-proof database of step history.
3. **Weights & Biases (WandB)**: Syncs training curves, learning rates, gradient norms, and hardware metrics to a cloud dashboard.

11. Deterministic State Checkpointing

Pretraining runs can take days or weeks, making them vulnerable to power outages, thermal throttling, or hardware faults. We implement a checkpointing system that saves the entire state of the training run.

Checkpoint Payload

Every 100 steps, the pipeline saves:

- `model_state.pt` : The model's weights.
- `optimizer.pt` : The AdamW optimizer states (including the m_t and v_t moments).
- `scheduler.pt` : The current learning rate step index.

dataset shuffling remains deterministic if you need to resume training.

12. 10-Step Pilot Loop Specification (M5 Mac Testing)

Before running a long pretraining job on expensive production hardware, we run a **10-step pilot loop** on our local **32GB M5 Laptop** to verify the system's stability.

Pilot Objectives

1. **Verify MPS Backend Allocations:** Confirm that tensors are routed to the Metal Performance Shaders (`mps`) backend rather than falling back to CPU emulation.
2. **Monitor Memory Stability:** Ensure unified memory usage remains stable and does not grow linearly with each step (which would indicate a memory leak).
3. **Verify Sequence Packing Shapes:** Ensure that the packed input tokens and shifted target labels are aligned correctly.

13. Production Scaling Design: M3 Ultra (90-Core GPU) Execution

Once the pipeline is verified by the pilot run, it can scale to production on our **M3 Ultra 512GB (90-Core GPU)** hardware. We use Apple-specific optimizations to maximize performance:

Apple Silicon Optimizations

- **Zero-Copy Memory Access:** Since the M-series CPU and GPU share a single unified memory pool, we can access tensors on the GPU without copying them over a slow PCIe bus.
- **Unified Memory Allocations:** We configure PyTorch to allow the MPS backend to allocate up to 95% of our system's unified RAM: `os.environ["PYTORCH_MPS_HIGH_WATERMARK_RATIO"] = "0.0"`
- **Mixed-Precision Execution (`torch.autocast`):** Runs model layers in native `bfloat16` to double throughput and halve memory usage.

FlashAttention kernels directly on the MPS device.

14. Hugging Face Model Card Specifications

To share the pretrained model with the community, we publish a **Model Card** (`README.md`) at the root of the model repository.

Model Card Template

```
---
language:
- en
- py
license: gemma
tags:
- code-generation
- python
- gemma
- text-generation
model-index:
- name: Gemma4-Custom-Code-NLP-9B-Pilot
  results: []
datasets:
- bigcode/starcoderdata
- HuggingFaceFW/fineweb-edu
co2_eq_emissions:
  emissions: 184.2 # Recorded via CodeCarbon during the run
  source: "CodeCarbon Tracker"
  training_type: "pretraining from scratch"
  geographical_location: "Local (Grid intensity dependent)"
---

# Gemma4-Custom-Code-NLP-9B-Pilot
```

```
## Model Details
* **Developed by**: Custom Local Pipeline
* **Model Type**: Decoder-only Transformer
* **Language(s)**: English (Natural Language), Python (Programming Logic)
* **Base Tokenizer**: `google/gemma-4-9b` (256K Vocabulary)
* **Pretraining Weight Ratio**: 80% Python Code / 20% English Educational Text
* **Context Window**: 2048 Tokens
* **License**: Gemma Open Weights License

## Model Architecture Parameters
* **Layers ( $N_{\text{layers}}$ )**: 18
* **Hidden Size ( $d_{\text{model}}$ )**: 2048
* **Attention Heads ( $N_{\text{heads}}$ )**: 8
* **Key-Value Heads ( $N_{\text{kv\_heads}}$ )**: 4 (Grouped-Query Attention)
* **Intermediate Dimension ( $d_{\text{ff}}$ )**: 16,384 (SwiGLU)
* **Tied Embeddings**: True
```

15. System Verification & Risk Analysis

This section analyzes the primary failure modes of scratch pretraining and details our mitigation strategies.

Risk & Mitigation Protocols

1. Out-of-Memory (OOM) Crashes on Local Hardware

- **Risk:** Large vocabulary matrices and context windows can overflow VRAM.
- **Mitigation:** We use Gradient Accumulation to split large global batch sizes into small, single-sequence micro-batches ($B = 1$) during local pilot runs.

2. Loss Divergence / Exploding Gradients